

Problem 1

Fix the number of agents to be $N = 20$. Generate a connected graph using Erdos-Renyi model for N agents and some $p \in (0, 1)$. Assume random initial positions for all N agents in $\mathbb{R}^2$. Using formation control strategy, display your name in capital letters in $\mathbb{R}^2$, letter by letter in a sequence. For instance, my name is Venkat and this is how I would do:

- Starting from initial position vector $\mathbf{x}[0]$, agents should go to positions forming the letter **V**.
- Starting from positions corresponding to letter **V**, agents should go to positions corresponding to letter **E**.
- Continue this letter sequence, until agents go to letter **T**.

Record your name formation sequence as an animation video through your simulation, host it in your github and simply give just the Github link containing both the code and the video as your solution. Ensure that your Github repository is made public before submitting it. For people having big lengthy names, choose the corresponding name which people use to call you on a regular basis - not the full entire name. For instance, if a student's full name is Manoj Kumar Sahu and he goes commonly by simply Manoj, then he should simulate the sequence MANOJ.

Solution:

We fix the number of agents to $N = 20$ and choose the edge probability $p = 0.4$. A connected Erdős-Rényi graph is generated and used as the communication graph between the agents. The agents are initially placed at random positions in $\mathbb{R}^2$.

The name to be formed is

PUSHKAR.

Each letter is represented using a set of line segments, and 20 target positions are sampled along these segments. The agents are assigned to the target positions and are driven using a distributed formation-control law based on relative positions with respect to their neighbors.

The simulation is implemented in the following Python file:

```

1 import numpy as np
2 import networkx as nx
3 import matplotlib.pyplot as plt
4 import matplotlib.animation as animation
5
6 np.random.seed(18)
7
8 N,P,NAME=20,0.4,"PUSHKAR"
9 DT,T_LETTER=0.02,4.0
10 STEPS=int(T_LETTER/DT)
11 K_GAIN,SCALE=2.0,6.0
12 FPS,STRIDE=30,4
13
14 def generate_graph(n,p,seed=1):
15     while True:
16         G=nx.erdos_renyi_graph(n,p,seed=seed)
17         if nx.is_connected(G): return G
18         seed+=1

```

```

19
20 G=generate_graph(N,P)
21 Adj=nx.to_numpy_array(G)
22 print(f"Graph: N={N}, p={P}, edges={G.number_of_edges()}, connected={nx.
    is_connected(G)}")
23
24 x0=np.random.uniform(-SCALE,SCALE,(N,2))
25
26 LETTER_SEGMENTS={
27 "P": [((-0.6,-1),(-0.6,1)),((-0.6,1),(0.3,1)),((0.3,1),(0.6,.7)),((0.6,.7),(0.6,.3))
    ,((0.6,.3),(0.3,0)),((0.3,0),(-0.6,0))],
28 "U": [((-0.6,1),(-0.6,-.5)),((-0.6,-.5),(-0.3,-1)),((-0.3,-1),(0.3,-1)),((0.3,-1)
    ,(0.6,-.5)),((0.6,-.5),(0.6,1))],
29 "S": [((0.6,.8),(0.3,1)),((0.3,1),(-0.4,1)),((-0.4,1),(-0.6,.7)),((-0.6,.7),(-0.6,.2))
    ,((-0.6,.2),(0.4,-.2)),((0.4,-.2),(0.6,-.5)),((0.6,-.5),(0.4,-1)),((0.4,-1),(-0.4,-1)
    ),((-0.4,-1),(-0.6,-.8))],
30 "H": [((-0.6,-1),(-0.6,1)),((0.6,-1),(0.6,1)),((-0.6,0),(0.6,0))],
31 "K": [((-0.6,-1),(-0.6,1)),((-0.6,0),(0.6,1)),((-0.6,0),(0.6,-1))],
32 "A": [((-0.7,-1),(0,1)),((0,1),(0.7,-1)),((-0.4,0),(0.4,0))],
33 "R": [((-0.6,-1),(-0.6,1)),((-0.6,1),(0.3,1)),((0.3,1),(0.6,.7)),((0.6,.7),(0.6,.3))
    ,((0.6,.3),(0.3,0)),((0.3,0),(-0.6,0)),((0,0),(0.7,-1))]
34 }
35
36 def sample_segment(a,b,n):
37     a,b=np.array(a),np.array(b)
38     t=np.linspace(0,1,n)
39     return np.outer(1-t,a)+np.outer(t,b)
40
41 def letter_points(letter,n=N):
42     seg=LETTER_SEGMENTS[letter]
43     lengths=[np.linalg.norm(np.array(b)-a) for a,b in seg]
44     counts=[max(1,round(n*l/sum(lengths))) for l in lengths]
45
46     while sum(counts)>n:
47         i=np.argmax(counts)
48         if counts[i]>1: counts[i]-=1
49     while sum(counts)<n: counts[np.argmax(lengths)]+=1
50
51     return np.vstack([sample_segment(a,b,c) for (a,b),c in zip(seg,countes)])*
        SCALE
52
53 targets=[letter_points(c) for c in NAME]
54
55 def assign_targets(x,target):
56     remaining=list(range(N))
57     assigned=np.zeros_like(target)
58     for i in range(N):
59         d=np.linalg.norm(target[remaining]-x[i],axis=1)
60         assigned[i]=target[remaining.pop(np.argmin(d))]
61     return assigned
62
63 def formation_control(x,target):
64     dx=np.zeros_like(x)
65     for i in range(N):
66         nbr=np.where(Adj[i]>0)[0]
67         if len(nbr):
68             dx[i]=-K_GAIN*(((x[i]-x[nbr])-(target[i]-target[nbr]))).sum(axis=0))
69     return dx
70
71 trajectory=[x0.copy()]

```

```

72 x=x0.copy()
73
74 for letter,target in zip(NAME,targets):
75     print(f"Forming letter: {letter}")
76     target=assign_targets(x,target)
77     for _ in range(STEPS):
78         x+=DT*formation_control(x,target)
79         trajectory.append(x.copy())
80
81 trajectory=np.array(trajectory)
82 print(f"Simulation finished: {len(trajectory)} states")
83
84 # Animation
85 fig,ax=plt.subplots(figsize=(8,8))
86 margin=2
87 ax.set_xlim(trajectory[:,0].min()-margin,trajectory[:,0].max()+margin)
88 ax.set_ylim(trajectory[:,1].min()-margin,trajectory[:,1].max()+margin)
89 ax.set_aspect("equal")
90 ax.set_title(f"Formation Control: {NAME} ({N} agents)")
91
92 scat=ax.scatter([],[],s=70,c="crimson",edgecolors="black",linewidths=.5,zorder
    =2)
93 edges=[ax.plot([],[],color="gray",lw=.5,alpha=.5)[0] for _ in G.edges()]
94 label=ax.text(.02,.96,"",transform=ax.transAxes,fontsize=16,fontweight="bold",va
    ="top")
95
96 frames=np.arange(0,len(trajectory),STRIDE)
97
98 def current_letter(step):
99     return NAME[min(step//STEPS,len(NAME)-1)]
100
101 def init():
102     scat.set_offsets(np.zeros((N,2)))
103     for e in edges: e.set_data([],[])
104     label.set_text("")
105     return [scat,label]+edges
106
107 def update(f):
108     step=frames[f]
109     pos=trajectory[step]
110     scat.set_offsets(pos)
111     for e,(i,j) in zip(edges,G.edges()):
112         e.set_data(pos[[i,j],0],pos[[i,j],1])
113     label.set_text(f"Target letter: {current_letter(step)}")
114     return [scat,label]+edges
115
116 anim=animation.FuncAnimation(fig,update,frames=len(frames),init_func=init,
117                             blit=True,interval=1000/FPS)
118
119 try:
120     anim.save("pushkar_formation_control.mp4",
121             writer=animation.FFMpegWriter(fps=FPS,bitrate=1800))
122     print("Saved: pushkar_formation_control.mp4")
123 except Exception as e:
124     print(f"FFmpeg unavailable: {e}")
125     anim.save("pushkar_formation_control.gif",
126             writer=animation.PillowWriter(fps=FPS))
127     print("Saved: pushkar_formation_control.gif")
128 plt.close(fig)
129

```

```

130 # Graph topology
131 fig,ax=plt.subplots(figsize=(6,6))
132 pos=nx.spring_layout(G,seed=1)
133 nx.draw(G,pos,ax=ax,with_labels=True,node_color="crimson",
134         edge_color="gray",font_color="white",font_size=8)
135 ax.set_title(f"Erdos-Renyi Graph (N={N}, p={P})")
136 fig.savefig("erdos_renyi_graph.png",dpi=150,bbox_inches="tight")
137 plt.close(fig)
138 print("Saved: erdos_renyi_graph.png")

```

The resulting formation sequence is recorded as an animation. The following files are hosted in the public GitHub repository:

- **Python Code:** <https://github.com/GPushkar1611/MAS-Assignments/blob/main/ass-3/P1.py>
- **Animation Video:** https://github.com/GPushkar1611/MAS-Assignments/blob/main/ass-3/pushkar_formation_control.mp4

The complete simulation demonstrates the agents transitioning from their random initial positions through the sequence

$$\mathbf{P} \rightarrow \mathbf{U} \rightarrow \mathbf{S} \rightarrow \mathbf{H} \rightarrow \mathbf{K} \rightarrow \mathbf{A} \rightarrow \mathbf{R}.$$

Problem 2

Suppose that there is a target intruder which has to be tracked by a swarm of $N \in \mathbb{N}$ drones. Each of the N drones are equipped with a sensor and the drones are connected by a communication network $\mathcal{G}(\mathcal{V}, \mathcal{E})$. The initial location of the intruder is denoted by $z_0 \in \mathbb{R}^3$ is not known exactly and $z_0 \sim \mathcal{N}(\bar{z}_0, \Sigma_0)$, where mean $\bar{z}_0 \in \mathbb{R}^3$ and covariance $\Sigma_0 \in \mathbb{R}^{3 \times 3}$ and is positive semi-definite.

The intruder target is doing a random walk with dynamics given by

$$z(t+1) = z(t) + w, \quad \forall t \in \mathbb{N}$$

where $w \sim \mathcal{N}(0, \Sigma_w)$ denotes the process noise (like the wind disturbance). Each sensor i is located at $x_i \in \mathbb{R}^3$ and assume that sensor i mounted on drone i knows its location x_i , and its measurement data $y_i(t)$ is given by the model $\forall t \in \mathbb{N}$ as

$$y_i(t) = x_i - z(t) + v_i$$

Here v_i is a random sensor measurement noise with $v_i \sim \mathcal{N}(0, \Sigma_v^{(i)})$, where $\Sigma_v^{(i)} \succeq 0$ is also known to agent i . We assume that noises for different sensors (v_i, v_j with $i, j \in 1, \dots, N$ and $i \neq j$) are independent. Now let each drone with their respective sensor $i = 1, \dots, N$ take one measurement using (2) and obtain $y_i(t)$ for each time $t = [0, T_{\max}]$. Do the following:

1. Formulate the distributed state estimation problem of the intruder's location $z(t)$ as a consensus optimization problem, and specify each local objective function f_i .
2. Find the estimate of the intruder's state $\hat{z}(t)$ using distributed state estimation technique (distributed gradient descent or distributed gradient tracking) with the data collected by all the sensors $\{y_i(t), t \in [0, T_{\max}]\}_{i=1}^N$ via (2).
3. Assume and fix the number of drones $N \in \mathbb{N}$ (typically less than 20) and also generate a random graph $\mathcal{G}(\mathcal{V}, \mathcal{E})$ with N agents (using Erdos-Renyi model) containing a spanning tree (so that graph is connected) and plot it. Assume and fix the random quantities for $\bar{z}_0, \Sigma_0, \Sigma_w, \{x_i, \Sigma_v^{(i)}\}_{i=1}^N$ depicting above figure. Simulate and plot the fixed sensor (that is, drone) locations and actual true intruder location trajectory using (1) at every time for $t = [0, T_{\max}]$. Assume yourself some $T_{\max} \in \mathbb{N}$. Plot the computed estimate of the intruder's state trajectory $\hat{z}(t)$ for all $t = [0, T_{\max}]$

and check if your estimate matches well with the actual intruder state $z(t)$ and report the estimation error ($e(t) = \hat{z}(t) - z(t)$) too.

Solution:

Part 1: Distributed State Estimation Formulation

From the measurement model

$$y_i(t) = x_i - z(t) + v_i$$

we can rewrite the measurement as $x_i - y_i(t) = z(t) - v_i$. Thus, sensor i obtains a noisy observation of the intruder state $z(t)$. Since

$$v_i \sim \mathcal{N}\left(0, \Sigma_v^{(i)}\right)$$

the local weighted least-squares objective at sensor i is

$$f_i(z, t) = \frac{1}{2} (z - x_i + y_i(t))^T \left(\Sigma_v^{(i)}\right)^\dagger (z - x_i + y_i(t))$$

Therefore, the centralized state estimation problem is

$$\hat{z}(t) = \arg \min_{z \in \mathbb{R}^3} \sum_{i=1}^N f_i(z, t)$$

To solve this in a distributed manner, each drone maintains a local estimate $z_i(t)$ and the problem is formulated as the consensus optimization problem

$$\begin{aligned} \min_{z_1, \dots, z_N \in \mathbb{R}^3} \quad & \sum_{i=1}^N f_i(z_i, t) \\ \text{subject to} \quad & z_i = z_j \quad \forall (i, j) \in \mathcal{E} \end{aligned}$$

At consensus, $z_1 = z_2 = \dots = z_N = \hat{z}(t)$, where the common value gives the distributed estimate of the intruder's location.

Part 2: Distributed State Estimation

Let $z_i^k(t) \in \mathbb{R}^3$ denote the local estimate maintained by drone i at iteration k . The gradient of the local objective is

$$\nabla f_i(z_i^k(t), t) = \left(\Sigma_v^{(i)}\right)^\dagger \left(z_i^k(t) - x_i + y_i(t)\right)$$

Using distributed gradient tracking, each drone updates its estimate according to

$$z_i^{k+1}(t) = \sum_{j=1}^N a_{ij} z_j^k(t) - \alpha s_i^k(t)$$

where $A = [a_{ij}]$ is a doubly stochastic mixing matrix associated with the communication graph and $\alpha > 0$ is the step size.

The gradient-tracking variable is updated as

$$s_i^{k+1}(t) = \sum_{j=1}^N a_{ij} s_j^k(t) + \nabla f_i(z_i^{k+1}(t), t) - \nabla f_i(z_i^k(t), t)$$

The variables are initialized as

$$\begin{aligned} z_i^0(t) &= x_i - y_i(t) \\ s_i^0(t) &= \nabla f_i(z_i^0(t), t) \end{aligned}$$

After sufficient iterations, the local estimates reach consensus, $z_1^k(t) \approx z_2^k(t) \approx \dots \approx z_N^k(t)$, and the intruder's estimated state is taken as

$$\hat{z}(t) = \frac{1}{N} \sum_{i=1}^N z_i^K(t)$$

where K is the final gradient-tracking iteration.

Thus, the above procedure is repeated for every $t = 0, 1, \dots, T_{\max}$ using the measurements collected by all sensors.

Part 3: Simulation and Results

We fix $N = 10$ drones and $T_{\max} = 50$. A connected Erdős–Rényi graph is generated with edge probability $p = 0.3$. The drone locations are randomly generated in $[-5, 5]^3$. We use

$$\bar{z}_0 = [0, 0, 0]^T$$

$$\Sigma_0 = I_3$$

$$\Sigma_w = 0.08I_3$$

$$\Sigma_v^{(i)} = 0.3I_3 \quad i = 1, \dots, N$$

The true intruder trajectory is generated using the random-walk model $z(t+1) = z(t) + w(t)$ and the measurements are generated according to $y_i(t) = x_i - z(t) + v_i(t)$. For each time step, the distributed gradient tracking algorithm from

Part 2 is run for $K = 150$ iterations with step size $\alpha = 0.05$.

The generated communication graph is shown below.

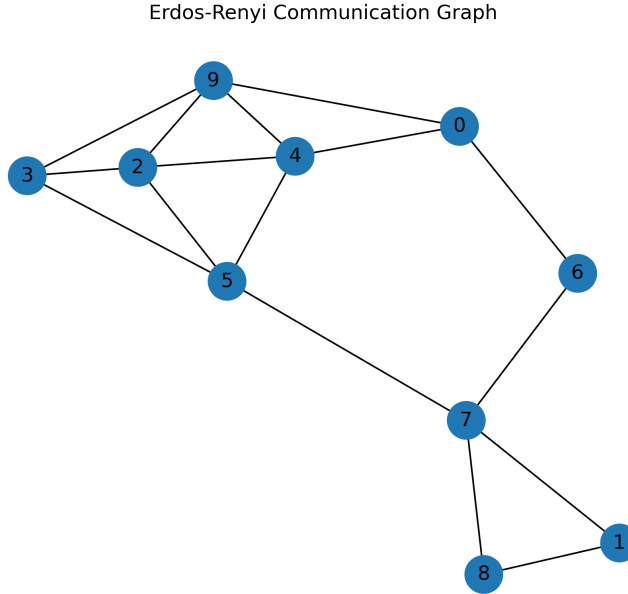


Figure 1: Connected Erdős–Rényi communication graph.

The true and estimated intruder trajectories, together with the fixed drone locations, are shown in Fig. 2.

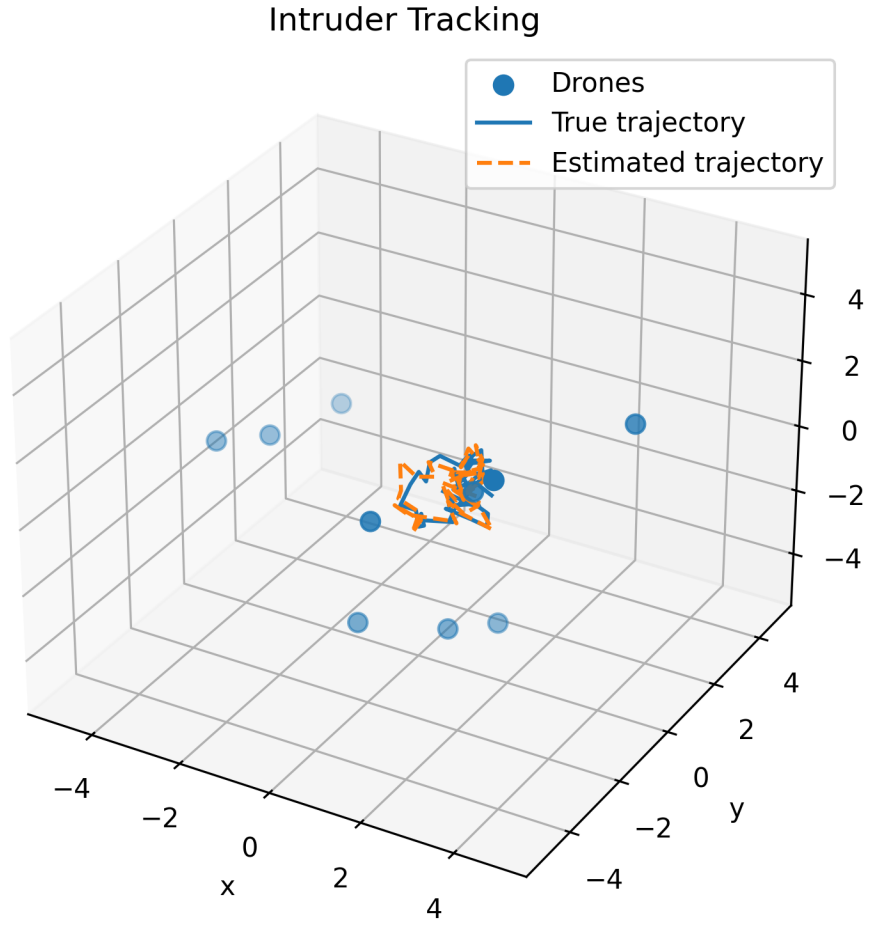


Figure 2: True and estimated intruder trajectories with drone locations.

The estimation error is defined as

$$e(t) = \hat{z}(t) - z(t)$$

and its norm over time is shown in Fig. 3.

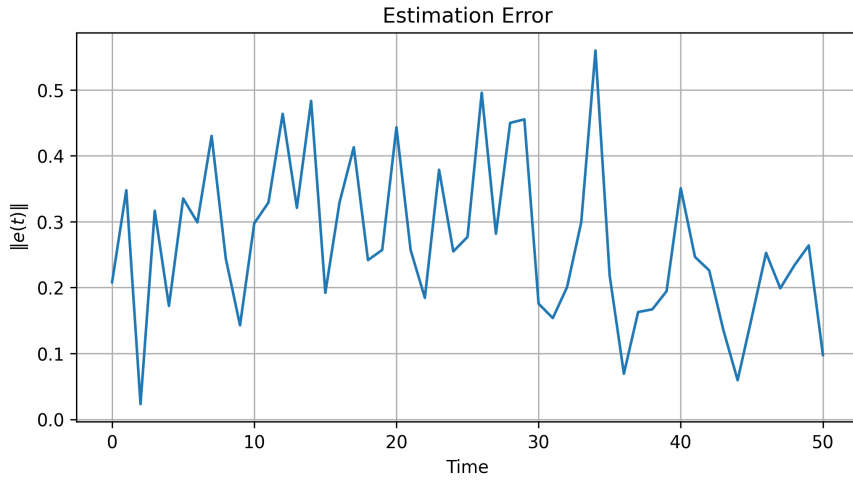


Figure 3: Norm of the estimation error $\|e(t)\|$.

The final and mean estimation errors obtained from the simulation are

$$\|e(T_{\max})\| = 0.0976$$

$$\frac{1}{T_{\max} + 1} \sum_{t=0}^{T_{\max}} \|e(t)\| = 0.2695$$

The estimated trajectory follows the actual intruder trajectory, with a final estimation error of approximately 0.0976 and a mean estimation error of approximately 0.2695. This indicates that the distributed state estimation scheme is able to track the moving intruder using measurements shared through the communication network.

The complete Python implementation is given below.

```

1 import numpy as np
2 import networkx as nx
3 import matplotlib.pyplot as plt
4 from pathlib import Path
5
6 np.random.seed(42)
7
8 N,Tmax=10,50
9 p=0.3
10 dim=3
11 alpha=0.05
12 K=150
13
14 # Connected Erd s -R nyi graph
15 while True:
16     G=nx.erdos_renyi_graph(N,p,seed=np.random.randint(10000))
17     if nx.is_connected(G):
18         break
19
20 A=np.zeros((N,N))
21 for i in range(N):
22     for j in range(N):
23         if i!=j and G.has_edge(i,j):
24             A[i,j]=1/(1+max(G.degree(i),G.degree(j)))
25     A[i,i]=1-A[i].sum()
26
27 # Initial state and noise covariances
28 zbar0=np.array([0.,0.,0.])
29 Sigma0=np.diag([1.,1.,1.])
30 Sigmax=np.diag([0.08,0.08,0.08])
31 Sigmay=[np.diag([0.3,0.3,0.3]) for _ in range(N)]
32
33 # Fixed drone locations
34 X=np.random.uniform(-5,5,(N,3))
35
36 # True intruder trajectory
37 z=np.zeros((Tmax+1,dim))
38 z[0]=np.random.multivariate_normal(zbar0,Sigma0)
39 for t in range(Tmax):
40     z[t+1]=z[t]+np.random.multivariate_normal(np.zeros(dim),Sigmax)
41
42 # Measurements
43 Y=np.zeros((Tmax+1,N,dim))
44 for t in range(Tmax+1):
45     for i in range(N):
46         v=np.random.multivariate_normal(np.zeros(dim),Sigmay[i])

```

```

47         Y[t,i]=X[i]-z[t]+v
48
49 # Distributed gradient tracking
50 zhat=np.zeros((Tmax+1,dim))
51
52 for t in range(Tmax+1):
53     zi=X-Y[t]
54     s=np.zeros((N,dim))
55
56     for i in range(N):
57         Rinv=np.linalg.pinv(Sigmav[i])
58         s[i]=Rinv@(zi[i]-X[i]+Y[t,i])
59
60     for _ in range(K):
61         znew=A@zi-alpha*s
62         snew=np.zeros((N,dim))
63
64         for i in range(N):
65             Rinv=np.linalg.pinv(Sigmav[i])
66             grad=Rinv@(znew[i]-X[i]+Y[t,i])
67             oldgrad=Rinv@(zi[i]-X[i]+Y[t,i])
68             snew[i]=grad-oldgrad
69
70         s=A@s+snew
71         zi=znew
72
73     zhat[t]=zi.mean(axis=0)
74
75 error=zhat-z
76 error_norm=np.linalg.norm(error,axis=1)
77
78 # Communication graph
79 pos=nx.spring_layout(G,seed=42)
80 plt.figure(figsize=(6,5))
81 nx.draw(G,pos,with_labels=True,node_size=500)
82 plt.title("Erdos-Renyi Communication Graph")
83 plt.savefig("communication_graph.png",dpi=300,bbox_inches="tight")
84 plt.show()
85
86 # 3D trajectories
87 fig=plt.figure(figsize=(8,6))
88 ax=fig.add_subplot(111,projection="3d")
89 ax.scatter(X[:,0],X[:,1],X[:,2],s=50,label="Drones")
90 ax.plot(z[:,0],z[:,1],z[:,2],label="True trajectory")
91 ax.plot(zhat[:,0],zhat[:,1],zhat[:,2],"--",label="Estimated trajectory")
92 ax.set_xlabel("x")
93 ax.set_ylabel("y")
94 ax.set_zlabel("z")
95 ax.set_title("Intruder Tracking")
96 ax.legend()
97 plt.savefig("intruder_tracking.png",dpi=300,bbox_inches="tight")
98 plt.show()
99
100 # Estimation error
101 plt.figure(figsize=(8,4))
102 plt.plot(error_norm)
103 plt.xlabel("Time")
104 plt.ylabel(r"$\|e(t)\|$")
105 plt.title("Estimation Error")
106 plt.grid()

```

```

107 plt.savefig("estimation_error.png",dpi=300,bbox_inches="tight")
108 plt.show()
109
110 print("Final estimation error:",error_norm[-1])
111 print("Mean estimation error:",error_norm.mean())

```

Problem 3

There are $N \in \mathbb{N}$ agents connected via an undirected connected graph $\mathcal{G}(\mathcal{V}, \mathcal{E})$. They must collaboratively execute $M \in \mathbb{N}$ tasks where $M > N$. The set of all tasks is denoted by $\mathcal{T} := \{T_1, \dots, T_M\}$. Each agent $i \in \mathcal{V}$ is assigned a local cost C_{ij} for performing a task $T_j \in \mathcal{T}$ and thereby giving us a cost matrix $\mathcal{C} \in \mathbb{R}^{N \times M}$. Further, each agent $i \in \mathcal{V}$ has a capacity constraint $b_i \in \mathbb{Z}_{\geq 0}$ which captures how many tasks that agent $i \in \mathcal{V}$ can take. Since, there is totally M tasks and each task should be assigned only once, we see that $\sum_{i=1}^N b_i = M$. Do the following:

1. Formulate the task allocation problem as an optimization problem with appropriate cost function and constraints. Your understanding of the problem and the way you formulate should be highlighted well. Give the ADMM updates for every agent $i \in \mathcal{V}$.

Hint: You should get a binary linear program (which is NP-hard to solve for large M, N). Hence, you will need to do convex relaxation which you can learn by yourself or come ask me.

2. Randomly choose the number of tasks M and number of agents N such that $M > N$. Fix them and generate an undirected connected graph $\mathcal{G}(\mathcal{V}, \mathcal{E})$ for the communication model (use Erdos-Renyi model). Randomly choose and fix the capacities b_i for every agent $i \in \mathcal{V}$ such that $\sum_{i=1}^N b_i = M$. Generate a random cost matrix $\mathcal{C} \in \mathbb{R}^{N \times M}$ and ensure each element $C_{ij} > 0$. Plot the graph $\mathcal{G}(\mathcal{V}, \mathcal{E})$ and the result of task allocation using ADMM simulation.

Fix the same (M, N) , play with different generated cost matrix $\mathcal{C} \in \mathbb{R}^{N \times M}$ and plot the resulting task allocation and explain your observations.

Solution:

Part 1: Task Allocation Formulation and ADMM

Let

$$x_{ij} = \begin{cases} 1, & \text{if agent } i \text{ is assigned task } T_j \\ 0, & \text{otherwise} \end{cases}$$

Let $X = [x_{ij}] \in \{0, 1\}^{N \times M}$ denote the task allocation matrix. The total allocation cost is

$$\sum_{i=1}^N \sum_{j=1}^M C_{ij} x_{ij}$$

Since each agent can perform b_i tasks and every task must be assigned exactly once, the binary linear program is

$$\begin{aligned}
\min_X \quad & \sum_{i=1}^N \sum_{j=1}^M C_{ij} x_{ij} \\
\text{s.t.} \quad & \sum_{j=1}^M x_{ij} = b_i & i = 1, \dots, N, \\
& \sum_{i=1}^N x_{ij} = 1 & j = 1, \dots, M, \\
& x_{ij} \in \{0, 1\} & \forall i, j
\end{aligned}$$

Since the binary constraint is non-convex, we use the convex relaxation $0 \leq x_{ij} \leq 1$. Thus, the relaxed problem is a linear program. For a distributed formulation, agent i maintains a local copy $X_i \in \mathbb{R}^{N \times M}$ of the allocation matrix. Define the local objective

$$f_i(X_i) = \sum_{j=1}^M C_{ij} x_{ij}^{(i)} + \mathcal{I}_{\mathcal{X}_i}(X_i)$$

where $\mathcal{I}_{\mathcal{X}_i}$ is the indicator function of the local feasible set

$$\mathcal{X}_i = \left\{ X_i : \begin{array}{l} 0 \leq x_{kj}^{(i)} \leq 1, \quad \forall k, j \\ \sum_{j=1}^M x_{ij}^{(i)} = b_i \\ \sum_{k=1}^N x_{kj}^{(i)} = 1 \quad \forall j \end{array} \right\}$$

The distributed problem is then formulated as

$$\begin{aligned} \min_{X_1, \dots, X_N} \quad & \sum_{i=1}^N f_i(X_i) \\ \text{s.t.} \quad & X_i = X_j, \quad \forall (i, j) \in \mathcal{E} \end{aligned}$$

The consensus constraint ensures that all agents eventually obtain the same allocation matrix.

For each edge $(i, j) \in \mathcal{E}$, introduce an auxiliary variable Z_{ij} and scaled dual variables $U_{ij}^{(i)}$ and $U_{ij}^{(j)}$. The ADMM updates at iteration k are

$$X_i^{k+1} = \arg \min_{X_i \in \mathcal{X}_i} \left[\sum_{j=1}^M C_{ij} x_{ij}^{(i)} + \frac{\rho}{2} \sum_{j \in \mathcal{N}_i} \|X_i - Z_{ij}^k + U_{ij}^{(i),k}\|_F^2 \right]$$

where $\mathcal{N}_i$ denotes the neighbors of agent i .

For every edge $(i, j) \in \mathcal{E}$, the auxiliary variable is updated as

$$Z_{ij}^{k+1} = \frac{1}{2} \left(X_i^{k+1} + U_{ij}^{(i),k} + X_j^{k+1} + U_{ij}^{(j),k} \right)$$

The dual variables are then updated by

$$\begin{aligned} U_{ij}^{(i),k+1} &= U_{ij}^{(i),k} + X_i^{k+1} - Z_{ij}^{k+1} \\ U_{ij}^{(j),k+1} &= U_{ij}^{(j),k} + X_j^{k+1} - Z_{ij}^{k+1} \end{aligned}$$

After convergence,

$$X_1 = X_2 = \dots = X_N = X^*$$

and the entries of X^* give the final task allocation. For the relaxed problem, an entry $x_{ij}^* = 1$ indicates that task T_j is assigned to agent i .

Part 2: ADMM Simulation

We choose

$$N = 4, \quad M = 8, \quad K = 100,$$

and generate a connected Erdős–Rényi graph with edge probability $p = 0.6$. The capacities are fixed as

$$(b_1, b_2, b_3, b_4) = (1, 2, 2, 3)$$

which satisfies $\sum_{i=1}^4 b_i = 8 = M$. The communication graph generated for the simulation is shown in Fig. 4.

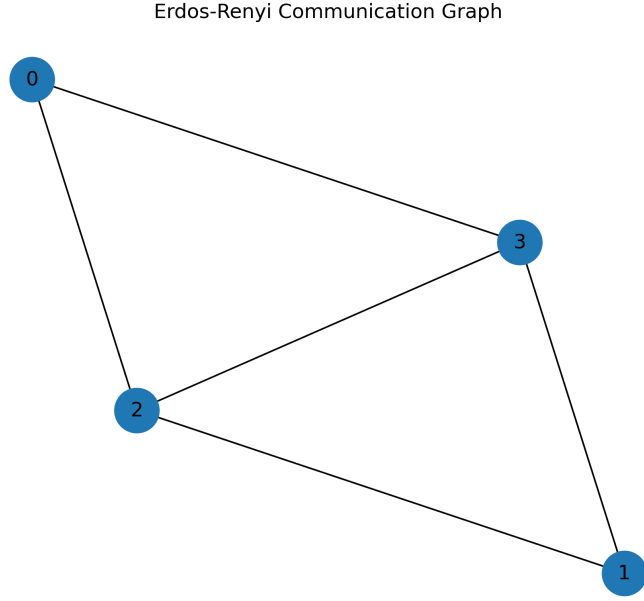


Figure 4: Connected Erdős–Rényi communication graph.

A positive cost matrix $\mathcal{C}_1$ is randomly generated and the ADMM algorithm is applied to the relaxed task allocation problem. The resulting allocation is

$$X_1 = \begin{bmatrix} 0 & 0 & 0 & 0 & 0 & 0 & 1 & 0 \\ 0 & 0 & 1 & 0 & 0 & 1 & 0 & 0 \\ 0 & 1 & 0 & 1 & 0 & 0 & 0 & 0 \\ 1 & 0 & 0 & 0 & 1 & 0 & 0 & 1 \end{bmatrix}$$

Thus, every task is assigned exactly once and the number of tasks assigned to each agent satisfies its capacity constraint. The corresponding total cost is

$$\sum_{i=1}^N \sum_{j=1}^M C_{ij} x_{ij} = 26.5053$$

The resulting task allocation is shown in Fig. 5.

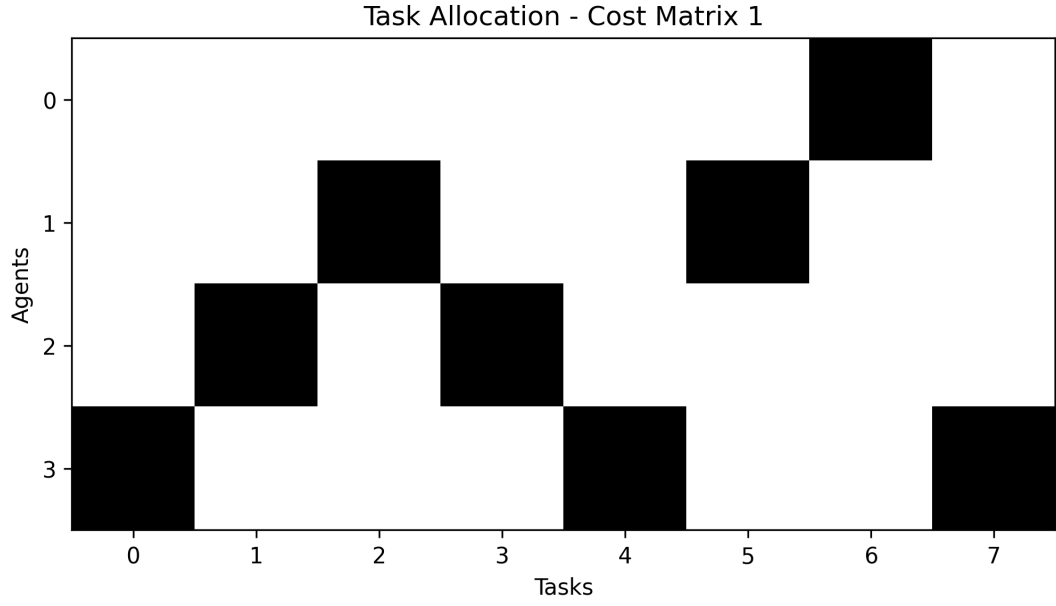


Figure 5: Task allocation for the first cost matrix.

Keeping the same M , N , graph, and capacities, a different positive cost matrix C_2 is generated. The resulting allocation is

$$X_2 = \begin{bmatrix} 0 & 0 & 0 & 0 & 1 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 & 0 & 0 & 1 \\ 0 & 1 & 0 & 1 & 0 & 0 & 0 & 0 \\ 1 & 0 & 0 & 0 & 0 & 1 & 1 & 0 \end{bmatrix}$$

with total cost

$$\sum_{i=1}^N \sum_{j=1}^M C_{ij} x_{ij} = 31.1770$$

The corresponding allocation is shown in Fig. 6.

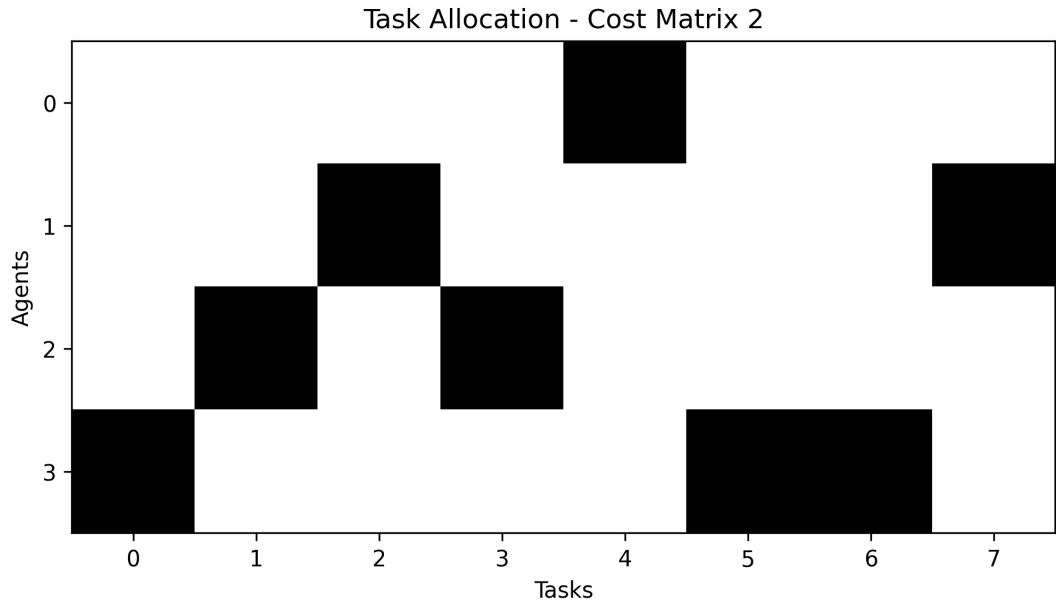


Figure 6: Task allocation for the second cost matrix.

The two simulations use the same communication graph and agent capacities, but produce different task allocations because the local costs of performing each task are different. In both cases, every task is assigned exactly once and every agent satisfies its capacity constraint. Thus, the cost matrix determines the preferred allocation among the feasible assignments, while the capacity constraints determine how many tasks each agent can receive.

The complete Python implementation is given below.

```

1 import numpy as np
2 import networkx as nx
3 import matplotlib.pyplot as plt
4 from scipy.optimize import minimize
5 from pathlib import Path
6
7 np.random.seed(42)
8
9 N,M=4,8
10 p=0.6
11 rho=1.0
12 K=100
13
14 # Connected Erdos-Renyi graph
15 while True:
16     G=nx.erdos_renyi_graph(N,p,seed=np.random.randint(10000))
17     if nx.is_connected(G):
18         break
19
20 # Capacities
21 b=np.array([1,2,2,3])
22
23 # Cost matrices
24 C1=np.random.uniform(1,10,(N,M))
25 C2=np.random.uniform(1,10,(N,M))
26
27 def edge_key(i,j):
28     return tuple(sorted((i,j)))
29
30 def solve_local(i,C,Z,U):
31     def objective(v):
32         X=v.reshape(N,M)
33         cost=C[i]@X[i]
34         penalty=0
35         for j in G.neighbors(i):
36             key=edge_key(i,j)
37             penalty+=np.sum((X-Z[key]+U[key][i])**2)
38         return cost+rho/2*penalty
39
40     cons=[{'type':'eq',
41           'fun':lambda v:v.reshape(N,M)[i].sum()-b[i]}]
42
43     for j in range(M):
44         cons.append({
45             'type':'eq',
46             'fun':lambda v,j=j:v.reshape(N,M)[: ,j].sum()-1
47         })
48
49     x0=np.ones((N,M))/N
50
51     result=minimize(
52         objective,x0.ravel(),
53         bounds=[(0,1)]*(N*M),

```

```

54         constraints=cons,
55         method='SLSQP',
56         options={'maxiter':300,'ftol':1e-8}
57     )
58
59     if not result.success:
60         print(f"Warning: local optimization failed for agent {i}: {result.
61             message}")
62
63     return result.x.reshape(N,M)
64
65 def admm(C):
66     X=[np.ones((N,M))/N for _ in range(N)]
67     Z={}
68     U={}
69
70     for i,j in G.edges:
71         key=edge_key(i,j)
72         Z[key]=np.ones((N,M))/N
73         U[key]={i:np.zeros((N,M)),j:np.zeros((N,M))}
74
75     for _ in range(K):
76         Xnew=[]
77
78         for i in range(N):
79             Xnew.append(solve_local(i,C,Z,U))
80
81         X=Xnew
82
83         for i,j in G.edges:
84             key=edge_key(i,j)
85             Z[key]=0.5*(X[i]+U[key][i]+X[j]+U[key][j])
86
87         for i,j in G.edges:
88             key=edge_key(i,j)
89             U[key][i]+=X[i]-Z[key]
90             U[key][j]+=X[j]-Z[key]
91
92     Xavg=np.mean(X,axis=0)
93     return Xavg
94
95 def get_allocation(X):
96     A=np.zeros((N,M),dtype=int)
97     remaining=b.copy()
98
99     for j in np.argsort(np.min(X,axis=0)):
100         agents=np.argsort(-X[:,j])
101         for i in agents:
102             if remaining[i]>0:
103                 A[i,j]=1
104                 remaining[i]-=1
105                 break
106
107     return A
108
109 # ADMM
110 X1=admm(C1)
111 X2=admm(C2)
112 A1=get_allocation(X1)

```

```

113 A2=get_allocation(X2)
114
115 # Graph
116 pos=nx.spring_layout(G,seed=42)
117 plt.figure(figsize=(6,5))
118 nx.draw(G,pos,with_labels=True,node_size=700)
119 plt.title("Erdos-Renyi Communication Graph")
120 plt.savefig("problem3_graph.png",dpi=300,bbox_inches="tight")
121 plt.show()
122
123 # Allocation 1
124 fig,ax=plt.subplots(figsize=(8,4))
125 ax.imshow(A1,cmap="Greys",vmin=0,vmax=1)
126 ax.set_xlabel("Tasks")
127 ax.set_ylabel("Agents")
128 ax.set_title("Task Allocation - Cost Matrix 1")
129 ax.set_xticks(range(M))
130 ax.set_yticks(range(N))
131 plt.savefig("allocation_cost1.png",dpi=300,bbox_inches="tight")
132 plt.show()
133
134 # Allocation 2
135 fig,ax=plt.subplots(figsize=(8,4))
136 ax.imshow(A2,cmap="Greys",vmin=0,vmax=1)
137 ax.set_xlabel("Tasks")
138 ax.set_ylabel("Agents")
139 ax.set_title("Task Allocation - Cost Matrix 2")
140 ax.set_xticks(range(M))
141 ax.set_yticks(range(N))
142 plt.savefig("allocation_cost2.png",dpi=300,bbox_inches="tight")
143 plt.show()
144
145 print("Capacities:",b)
146 print("\nCost Matrix 1:\n",np.round(C1,2))
147 print("\nAllocation 1:\n",A1)
148 print("\nCost Matrix 2:\n",np.round(C2,2))
149 print("\nAllocation 2:\n",A2)
150 print("\nTotal cost 1:",np.sum(C1*A1))
151 print("Total cost 2:",np.sum(C2*A2))

```

Problem 4

Consider the LASSO problem

$$\min_x \left[\frac{1}{2} \|Ax - b\|^2 + \lambda \|x\|_1 \right]$$

Write the ADMM iteration updates by solving and writing the closed form solution for x, z updates.

Solution:

Introduce an auxiliary variable z and rewrite the LASSO problem as

$$\begin{aligned} \min_{x,z} \quad & \frac{1}{2} \|Ax - b\|^2 + \lambda \|z\|_1 \\ \text{s.t.} \quad & x = z \end{aligned}$$

Using the scaled dual variable u , the augmented Lagrangian is

$$L_\rho(x, z, u) = \frac{1}{2} \|Ax - b\|^2 + \lambda \|z\|_1 + \frac{\rho}{2} \|x - z + u\|^2$$

The ADMM updates are given by

x -update:

$$x^{k+1} = \arg \min_x \left\{ \frac{1}{2} \|Ax - b\|^2 + \frac{\rho}{2} \|x - z^k + u^k\|^2 \right\}$$

Setting the gradient to zero gives

$$A^T(Ax^{k+1} - b) + \rho(x^{k+1} - z^k + u^k) = 0$$

and hence the closed-form solution is

$$x^{k+1} = (A^T A + \rho I)^{-1} (A^T b + \rho(z^k - u^k))$$

z -update:

$$z^{k+1} = \arg \min_z \left\{ \lambda \|z\|_1 + \frac{\rho}{2} \|x^{k+1} - z + u^k\|^2 \right\}$$

This is the proximal operator of the ℓ_1 norm, giving $z^{k+1} = S_{\lambda/\rho}(x^{k+1} + u^k)$, where $S_\tau(\cdot)$ is the soft-thresholding operator defined component-wise as

$$S_\tau(v_i) = \text{sign}(v_i) \max(|v_i| - \tau, 0)$$

Dual update:

$$u^{k+1} = u^k + x^{k+1} - z^{k+1}$$

Therefore, the complete ADMM iteration is

$$x^{k+1} = (A^T A + \rho I)^{-1} (A^T b + \rho(z^k - u^k))$$

$$z^{k+1} = S_{\lambda/\rho}(x^{k+1} + u^k)$$

$$u^{k+1} = u^k + x^{k+1} - z^{k+1}$$

AI Usage Disclosure

The improvement in the formatting and presentation of this assignment was achieved using the AI technology. The mathematical representation, solutions, simulations, and results have been verified.